

PROTOCOLO UDP CONFIABLE PARA RED NEURONAL DISTRIBUIDA

Proyecto de Redes y Comunicaciones

28 de mayo de 2026

Índice

1. Descripción General	3
2. Configuración de la Red Neuronal	3
3. Arquitectura General	3
4. Responsabilidades del Sistema	4
4.1. Python	4
4.2. Servidor Principal	5
4.3. Workers	5
5. Transferencia Confiable de Datos (RDT)	6
6. Características del Protocolo RDT	6
7. Parámetros del Protocolo	7
8. Formato del Paquete UDP	7
8.1. Estructura del paquete	7
8.2. Descripción de campos	8
9. Tipos de Paquete	8
10. Tipos de Objeto	8
11. Capa de Aplicación y Objetos Serializados	8
12. Campos Usados por Cada Tipo de Paquete	9
13. Integridad de Paquetes	10
13.1. Si el CRC32 coincide	10
13.2. Si el CRC32 falla	10
14. Números de Secuencia, ACK y Ventana	11
15. Timeouts y Reintentos	11

16.Estados del Emisor y Receptor	11
16.1. Estados del Emisor	11
16.2. Estados del Receptor	11
17.Secuencia de Comunicación	11
18.Distribución del Batch y Objetos	13
19.Serialización de Objetos	14
20.Flujo General del Sistema	14
21.Promedio de Gradientes y Actualización de Pesos	15
22.Integración Python y C++	15
23.Posibles Mejoras Futuras	16
24.Tecnologías Utilizadas	16
25.Conclusión	16

1. Descripción General

El objetivo del proyecto es implementar un sistema de Red Neuronal Distribuida (DNN) donde el cálculo de backpropagation se reparte entre varias computadoras o terminales y el servidor principal combina los resultados mediante el promedio de gradientes.

- La red neuronal se define y entrena en Python.
- La ejecución principal se inicia desde un archivo `.py`.
- Python llama a una biblioteca distribuida en C++ mediante `pybind11`.
- La biblioteca C++ implementa la serialización de objetos, la comunicación UDP y el protocolo RDT.
- La comunicación entre nodos utiliza UDP.
- La confiabilidad se implementa manualmente mediante un protocolo RDT (Reliable Data Transfer) basado en Go-Back-N puro con ACK acumulativo.

La arquitectura del sistema está compuesta por:

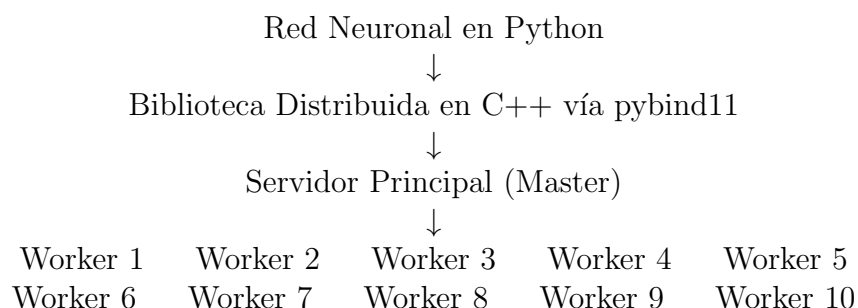
- Un servidor principal (Main Server / Master)
- Diez servidores trabajadores (Workers)
- Un programa o módulo de distribución de datos invocado por el servidor principal

El servidor principal toma un único batch global y lo divide en 11 partes iguales: una parte queda en el master y las otras 10 partes se envían a los workers. El forward pass se ejecuta únicamente en el master. Luego el master construye un objeto serializado `WORK_ASSIGNMENT` para cada worker, que contiene su partición del batch y todos los tensores necesarios para que pueda ejecutar backpropagation correctamente. Finalmente, recibe un objeto `GRADIENT_RESULT` desde cada worker, promedia los gradientes elemento a elemento y actualiza los pesos de la red neuronal principal.

2. Configuración de la Red Neuronal

La configuración de la red neuronal es la siguiente:

3. Arquitectura General



Parámetro	Valor
Capas ocultas	4
Neuronas por capa oculta	200
Matrices base de pesos	4 matrices completas
Tamaño de cada matriz oculta	200x200
Número de batches	1 batch global
División del batch	11 particiones
Épocas	1
Nodos de backpropagation	1 master + 10 workers
Objeto master → worker	WORK_ASSIGNMENT: partición del batch, pesos, activaciones, preactivaciones, etiquetas, derivadas necesarias y metadatos
Objeto worker → master	GRADIENT_RESULT: gradientes calculados por el worker y metadatos de reconstrucción
Protocolo de comunicación	UDP
Capa de confiabilidad	Go-Back-N RDT con ACK acumulativo
Tipo de dato principal	float32
Formato binario de tensores	little-endian
Endianness del protocolo	big-endian / network byte order
Tamaño de ventana inicial	8 paquetes
Timeout inicial	500 ms
Máximo de reintentos	5

4. Responsabilidades del Sistema

4.1. Python

Python será responsable de:

- Cargar el dataset
- Definir la red neuronal
- Ejecutar la lógica de entrenamiento de la red neuronal
- Seleccionar el único batch global de entrenamiento
- Ejecutar el forward pass únicamente en el master
- Preparar las particiones del batch y los tensores intermedios necesarios para backpropagation
- Construir los objetos `WORK_ASSIGNMENT` que serán enviados a los workers

- Ejecutar backpropagation local en el master sobre la partición 0
- Llamar a la biblioteca distribuida en C++ desde el archivo principal `.py`
- Recibir gradientes promediados
- Actualizar la red neuronal principal usando los gradientes combinados

La integración entre Python y C++ se realizará mediante `pybind11`.

4.2. Servidor Principal

El servidor principal será responsable de:

- Invocar el programa de distribución del batch
- Dividir el batch global en 11 partes iguales
- Mantener la partición 0 para el master
- Enviar las 10 particiones restantes a los workers
- Recibir desde Python los objetos `WORK_ASSIGNMENT` serializables
- Serializar cada objeto de aplicación como una secuencia binaria opaca para la capa RDT
- Construir paquetes UDP
- Administrar ventana Go-Back-N
- Enviar objetos serializados a los workers
- Administrar ACK y retransmisiones
- Recibir objetos `GRADIENT_RESULT` desde los workers
- Reentregar a Python los gradientes contenidos en los objetos reconstruidos
- Promediar los gradientes del master con los gradientes recibidos
- Retornar los gradientes promediados hacia Python para actualizar los pesos

4.3. Workers

Los workers serán responsables de:

- Recibir paquetes UDP
- Verificar integridad
- Reconstruir fragmentos en orden
- Detectar pérdidas y duplicados
- Descartar paquetes fuera de orden

- Recibir y reconstruir un objeto `WORK_ASSIGNMENT`
- Extraer de ese objeto su partición del batch, pesos, activaciones, preactivaciones, etiquetas, derivadas y metadatos necesarios
- Ejecutar backpropagation local sin realizar forward pass propio
- Calcular gradientes para su partición
- Retornar al master un objeto `GRADIENT_RESULT` con los gradientes calculados

5. Transferencia Confiable de Datos (RDT)

UDP no garantiza:

- Entrega de paquetes
- Orden correcto
- Integridad
- Protección contra duplicados

Por esta razón se implementará un protocolo RDT sobre UDP utilizando Go-Back-N.

6. Características del Protocolo RDT

El protocolo implementará el mecanismo Go-Back-N sobre UDP. Go-Back-N permitirá enviar múltiples paquetes consecutivos sin esperar una confirmación inmediata por cada uno de ellos. El protocolo no utilizará NACK explícitos; ante paquetes corruptos, duplicados o fuera de orden, el receptor reenviará el último ACK acumulativo válido y el emisor recuperará la transferencia mediante timeout y retransmisión Go-Back-N.

Característica	Descripción
ACK acumulativo	Confirma el último paquete recibido correctamente en orden
Ventana deslizante	Permite mantener varios paquetes en tránsito simultáneamente
Número de secuencia	Detecta pérdidas, duplicados y mantiene el orden
Timeout	Detecta pérdida de paquetes o ACKs
Retransmisión Go-Back-N	Retransmite todos los paquetes desde el paquete posterior al último ACK confirmado
CRC32	Detecta corrupción tanto en el encabezado como en el payload
Recepción en orden	El receptor solo acepta el paquete esperado y descarta paquetes fuera de orden

Parámetro	Valor
Endianness del header	Big-endian / network byte order
Tamaño máximo del datagrama UDP	512 bytes
Tamaño máximo del payload	476 bytes
Tamaño inicial de ventana	8 paquetes
Timeout inicial	500 ms
Máximo de reintentos	5
Primer número de secuencia DATA	0
ACK inicial sin paquetes válidos	ACK_NONE = 0xFFFFFFFF
CRC	CRC32 sobre header y payload válido

7. Parámetros del Protocolo

Los siguientes valores serán utilizados como configuración inicial del protocolo:

Todos los campos numéricos del encabezado del protocolo serán serializados en big-endian, también conocido como network byte order. El payload será tratado por la capa RDT como una secuencia opaca de bytes. La interpretación de tensores, dtype, dimensiones y metadatos pertenecerá a la capa de aplicación contenida dentro del objeto serializado.

Si un paquete o conjunto de paquetes supera el máximo de reintentos definido, la transferencia se considerará fallida y será cancelada.

8. Formato del Paquete UDP

Cada datagrama UDP tendrá un tamaño máximo de 512 bytes. El encabezado del protocolo tendrá tamaño fijo de 36 bytes y el payload podrá contener hasta 476 bytes del objeto serializado.

8.1. Estructura del paquete

Campo	Tamaño
Tipo de paquete	1 byte
Número de secuencia	4 bytes
Número ACK	4 bytes
ID de transferencia	4 bytes
ID del emisor	2 bytes
ID del receptor	2 bytes
Tipo de objeto	1 byte
Tamaño total del objeto	4 bytes
Número de fragmento	4 bytes
Total de fragmentos	4 bytes
Tamaño útil del payload	2 bytes
CRC32	4 bytes
Payload	476 bytes

8.2. Descripción de campos

- **Tipo de paquete:** Identifica si el paquete corresponde a DATA, ACK, START o END.
- **Número de secuencia:** Identifica el orden lógico del paquete dentro de la transferencia. Permite detectar pérdidas, duplicados y mantener el orden correcto.
- **Número ACK:** Indica el último número de secuencia recibido correctamente y en orden. Este campo se utiliza principalmente en paquetes ACK. Por ejemplo, si el receptor recibió correctamente los paquetes 0, 1 y 2, enviará un ACK con valor 2. Si todavía no recibió ningún DATA válido, utilizará el valor especial ACK_NONE.
- **ID de transferencia:** Identifica una transferencia lógica completa y actúa como ID del objeto serializado. Todos los fragmentos que pertenecen al mismo objeto utilizan el mismo ID de transferencia.
- **ID del emisor:** Identifica al nodo que envía la transferencia. El master utiliza el ID 0 y los workers utilizan IDs del 1 al 10.
- **ID del receptor:** Identifica al nodo destino. Este campo evita mezclar respuestas o asignaciones cuando varias transferencias se ejecutan en paralelo.
- **Tipo de objeto:** Define el tipo del objeto serializado transportado por la transferencia, por ejemplo WORK_ASSIGNMENT o GRADIENT_RESULT.
- **Tamaño total del objeto:** Indica el tamaño total en bytes del objeto serializado antes de la fragmentación. Permite validar la reconstrucción final.
- **Número de fragmento:** Indica la posición del fragmento dentro del objeto transmitido.
- **Total de fragmentos:** Permite reconstruir completamente el objeto original.
- **Tamaño útil del payload:** Indica cuántos bytes válidos contiene el payload. Este valor no puede superar 476 bytes.
- **CRC32:** Código utilizado para detectar corrupción en el encabezado y en el payload. Para calcularlo, el campo CRC32 se coloca temporalmente en cero y luego se calcula el CRC32 sobre todo el encabezado más el payload válido.
- **Payload:** Contiene un fragmento de bytes del objeto serializado. Para la capa RDT, este contenido es opaco.

9. Tipos de Paquete

10. Tipos de Objeto

11. Capa de Aplicación y Objetos Serializados

El protocolo RDT no interpretará tensores, pesos, activaciones, etiquetas ni gradientes. Su responsabilidad será transportar de forma confiable un objeto serializado completo, identificado por un único ID de transferencia.

Tipo	Valor numérico	Significado
DATA	0x01	Paquete de datos
ACK	0x02	ACK acumulativo
START	0x03	Inicio de transmisión
END	0x04	Fin de transmisión

Objeto	Valor numérico	Significado
WORK_ASSIGNMENT	0x01	Objeto enviado por el master a un worker. Contiene la partición del batch y todos los tensores necesarios para backpropagation.
GRADIENT_RESULT	0x02	Objeto enviado por un worker al master. Contiene los gradientes calculados y metadatos de reconstrucción.
CONTROL	0x03	Objeto de control para futuras extensiones del protocolo de aplicación.
NONE	0x00	Sin operación asociada

La semántica de los datos estará definida en una capa superior de aplicación:

- **WORK_ASSIGNMENT**: objeto creado por el master para un worker específico. Incluye la partición del batch global, pesos, activaciones, preactivaciones, etiquetas, derivadas de pérdida y metadatos necesarios para ejecutar backpropagation local.
- **GRADIENT_RESULT**: objeto creado por un worker para el master. Incluye los gradientes calculados sobre su partición y los metadatos necesarios para validar forma, dtype, capa y parámetro asociado.

Cada objeto será serializado por la biblioteca C++ en una secuencia binaria. Luego el protocolo RDT fragmentará esa secuencia en paquetes DATA, los transmitirá usando Go-Back-N y reconstruirá exactamente el mismo objeto en el receptor. Una vez reconstruido, la capa de aplicación deserializará el objeto y procesará sus tensores internos.

12. Campos Usados por Cada Tipo de Paquete

No todos los tipos de paquete utilizan todos los campos del encabezado. Los campos no utilizados deberán enviarse con valor cero.

Tipo de paquete	Campos principales utilizados
START	Tipo de paquete, ID de transferencia, ID del emisor, ID del receptor, tipo de objeto, tamaño total del objeto, total de fragmentos, CRC32
DATA	Tipo de paquete, número de secuencia, ID de transferencia, ID del emisor, ID del receptor, tipo de objeto, tamaño total del objeto, número de fragmento, total de fragmentos, tamaño útil del payload, CRC32, payload
ACK	Tipo de paquete, número ACK, ID de transferencia, ID del emisor, ID del receptor, CRC32

END	Tipo de paquete, número de secuencia, número ACK, ID de transferencia, ID del emisor, ID del receptor, CRC32
-----	--

13. Integridad de Paquetes

Cada paquete incluirá un valor CRC32 calculado sobre el encabezado y el payload válido del paquete.

Para realizar el cálculo, el campo CRC32 se coloca temporalmente en cero. Luego se calcula el CRC32 sobre:

$$CRC32 = CRC32(\text{header con } CRC32 = 0 + \text{payload válido})$$

Esto permite detectar corrupción no solo en el contenido transmitido, sino también en campos críticos como el tipo de paquete, número de secuencia, número ACK, ID de transferencia, ID del emisor, ID del receptor, tipo de objeto, tamaño total del objeto, número de fragmento, total de fragmentos y tamaño útil del payload.

El receptor recalculará el CRC32 utilizando el mismo procedimiento y lo comparará con el valor recibido.

13.1. Si el CRC32 coincide

- El paquete se considera íntegro.
- El receptor verifica si el número de secuencia corresponde al paquete esperado.
- Si el paquete es el esperado, se acepta y se actualiza el último número de secuencia recibido correctamente en orden.
- Se envía un ACK acumulativo indicando el último paquete recibido correctamente en orden.

13.2. Si el CRC32 falla

- El paquete se considera corrupto.
- El paquete completo se descarta.
- No se actualiza el número de secuencia esperado.
- El receptor reenvía el ACK acumulativo del último paquete recibido correctamente en orden. Si todavía no recibió ningún DATA válido, responde con ACK_NONE.
- No se envía NACK, porque el protocolo utiliza Go-Back-N puro.

14. Números de Secuencia, ACK y Ventana

Cada paquete DATA tendrá un número de secuencia único dentro de la transferencia.

El protocolo utilizará una ventana deslizante Go-Back-N de tamaño configurable. El valor inicial de la ventana será de 8 paquetes.

La ventana permitirá mantener múltiples paquetes en tránsito simultáneamente.

El campo ACK tendrá semántica acumulativa:

- Un ACK con valor n indica que todos los paquetes desde el inicio hasta el paquete n fueron recibidos correctamente y en orden.
- Un ACK con valor ACK_NONE indica que todavía no se recibió ningún paquete DATA válido de la transferencia.
- ACK_NONE es un valor centinela y no se interpreta como un número de secuencia mayor que los demás.
- Si el receptor espera el paquete k pero recibe un paquete distinto, lo descarta y vuelve a enviar el ACK del último paquete válido en orden.
- Si se pierde un ACK, el emisor podrá continuar si recibe un ACK posterior válido con un número mayor.

Los números de secuencia permitirán:

- Detectar paquetes duplicados
- Detectar paquetes perdidos
- Mantener el orden correcto
- Controlar retransmisiones
- Implementar ACK acumulativos

15. Timeouts y Reintentos

El protocolo utilizará un timeout inicial de 500 ms.

Si el emisor no recibe un ACK válido antes de que expire el timeout, retransmitirá todos los paquetes pendientes desde el paquete posterior al último ACK confirmado.

El número máximo de reintentos será 5. Si después de 5 reintentos no se recibe confirmación, la transferencia se marcará como fallida y será cancelada.

16. Estados del Emisor y Receptor

16.1. Estados del Emisor

16.2. Estados del Receptor

17. Secuencia de Comunicación

La transferencia confiable seguirá el siguiente flujo. El mismo procedimiento se utilizará en ambos sentidos: master a worker para enviar objetos WORK_ASSIGNMENT, y worker a

Estado	Descripción
IDLE	El emisor está esperando una nueva transferencia
START_SENT	El emisor envió START y espera ACK de inicio
SENDING	El emisor envía paquetes DATA dentro de la ventana Go-Back-N
WAITING_ACK	El emisor espera ACKs acumulativos de los paquetes enviados
END_SENT	El emisor envió END y espera ACK final
DONE	La transferencia terminó correctamente
FAILED	La transferencia falló por exceso de reintentos o error irrecuperable

Estado	Descripción
IDLE	El receptor espera un paquete START
RECEIVING	El receptor acepta únicamente el paquete DATA esperado y descarta paquetes fuera de orden
COMPLETE	El receptor recibió todos los fragmentos correctamente y espera o responde al paquete END
DONE	La transferencia terminó correctamente
FAILED	La transferencia fue cancelada por error de protocolo o fallo de comunicación

master para devolver objetos `GRADIENT_RESULT`. En cada transferencia, quien envía el objeto serializado actúa como emisor Go-Back-N y quien lo recibe actúa como receptor.

1. El emisor envía un paquete START.
2. El receptor responde con ACK.
3. El emisor comienza el envío continuo de paquetes DATA dentro de la ventana Go-Back-N.
4. El receptor verifica el CRC32 calculado sobre encabezado y payload.
5. Si el CRC32 es válido, el receptor verifica el número de secuencia.
6. Si el paquete recibido es exactamente el paquete esperado:
 - El receptor acepta el paquete.
 - Actualiza el último número de secuencia recibido correctamente en orden.
 - Envía un ACK acumulativo con ese número de secuencia.
7. Si el paquete recibido está duplicado o fuera de orden:
 - El receptor descarta el paquete.
 - El receptor reenvía un ACK acumulativo con el último número de secuencia recibido correctamente en orden.
 - Si todavía no existe un paquete válido confirmado, el receptor responde con `ACK_NONE`.

8. Si un paquete llega corrupto:
 - El receptor descarta el paquete completo.
 - El receptor no avanza su número de secuencia esperado.
 - El receptor reenvía el último ACK acumulativo válido o ACK_NONE si aún no recibió DATA válido.
9. Si ocurre timeout:
 - El emisor retransmite todos los paquetes desde el paquete posterior al último ACK confirmado.
 - Si el último ACK confirmado es ACK_NONE, el emisor retransmite desde el paquete DATA con secuencia 0.
10. Si se supera el máximo de reintentos:
 - El emisor cancela la transferencia.
 - El estado de la transferencia pasa a FAILED.
11. Cuando todos los fragmentos son enviados y confirmados:
 - El emisor envía END.
 - El receptor responde con ACK final.

18. Distribución del Batch y Objetos

El servidor principal coordinará la distribución del aprendizaje. Para ello invocará un programa o módulo de distribución que tomará un único batch global y lo dividirá en 11 partes iguales.

- La partición 0 será usada por el master.
- Las particiones 1 a 10 serán enviadas a los workers 1 a 10.
- Los workers podrán ejecutarse en computadoras distintas o en terminales separadas.

El forward pass se realizará únicamente en el master sobre el batch global. Durante ese forward pass, el master guardará los estados intermedios necesarios para backpropagation.

La red neuronal tendrá 4 capas ocultas de 200 neuronas. Por lo tanto, la implementación base utilizará 4 matrices principales de pesos:

$$W_1, W_2, W_3, W_4$$

Cada matriz tendrá tamaño 200x200. Sin embargo, el protocolo no se limita a enviar estas matrices. El master encapsulará en un objeto `WORK_ASSIGNMENT` todo dato requerido para ejecutar backpropagation de forma correcta sobre la partición del worker, incluyendo:

- La partición correspondiente del batch global.
- Las matrices de pesos requeridas por la red.

- Activaciones generadas por el forward pass del master.
- Preactivaciones necesarias para derivadas de funciones de activación.
- Etiquetas o targets de la partición.
- Derivadas iniciales de la función de pérdida, cuando corresponda.
- Metadatos de forma, dtype, capa, partición y reconstrucción.

Si la red utiliza biases u otros parámetros entrenables, estos se incluirán dentro del mismo objeto de aplicación y se tratarán con el mismo mecanismo de tensores y gradientes.

19. Serialización de Objetos

Los objetos de aplicación serán serializados como una secuencia binaria única. Dentro de cada objeto, las particiones del batch y los tensores se almacenarán como arreglos continuos en formato little-endian. El tipo de dato principal será float32, pero cada objeto incluirá metadatos que indiquen dtype, dimensiones, tipo de tensor, capa, parámetro asociado y partición.

Python enviará arreglos NumPy hacia C++ mediante pybind11.

C++ convertirá cada objeto en una secuencia binaria para fragmentarla en paquetes UDP de hasta 512 bytes.

Cada worker reconstruirá el objeto completo antes de deserializarlo e iniciar su cálculo local de backpropagation.

20. Flujo General del Sistema

1. Python carga el dataset y define la red neuronal.
2. Python selecciona un único batch global de entrenamiento.
3. El servidor principal invoca el programa de distribución del batch.
4. El batch global se divide en 11 partes iguales.
5. El master conserva la partición 0 y asigna las otras 10 particiones a los workers.
6. El master ejecuta el forward pass sobre el batch global completo.
7. El master guarda pesos, activaciones, preactivaciones, etiquetas, derivadas iniciales y metadatos necesarios.
8. Python construye un objeto `WORK_ASSIGNMENT` para cada worker y lo entrega a la biblioteca C++ mediante pybind11.
9. C++ serializa y transmite cada objeto `WORK_ASSIGNMENT` usando UDP con Go-Back-N.
10. Cada worker reconstruye y deserializa su objeto `WORK_ASSIGNMENT`.

11. Cada worker ejecuta backpropagation local sobre su partición sin realizar forward pass propio.
12. El master ejecuta backpropagation local sobre la partición 0.
13. Cada worker construye un objeto `GRADIENT_RESULT` con sus gradientes y llama a la biblioteca C++ para enviarlo al master.
14. El master reconstruye y deserializa los objetos `GRADIENT_RESULT` de los 10 workers.
15. El master promedia sus gradientes locales con los gradientes recibidos.
16. Python actualiza la red neuronal principal aplicando los gradientes promediados.

21. Promedio de Gradientes y Actualización de Pesos

La operación principal de combinación será el promedio de gradientes por capa. Para cada capa oculta l , el master calculará:

$$G_l^{avg} = \frac{G_l^{master} + \sum_{i=1}^{10} G_l^{worker_i}}{11}$$

Este promedio se realizará para los gradientes asociados a las 4 matrices principales de pesos:

$$l \in \{1, 2, 3, 4\}$$

Luego, Python actualizará los pesos de la red principal usando una regla de optimización. En el caso de SGD simple:

$$W_l^{new} = W_l^{old} - \eta G_l^{avg}$$

Si la implementación incluye biases u otros parámetros entrenables, el mismo esquema se aplicará a sus gradientes correspondientes.

22. Integración Python y C++

El proyecto utilizará pybind11 para conectar Python con C++.

Ejemplo:

```
import distributed_nn

pesos_actualizados = distributed_nn.run_training_round(
    dataset,
    pesos_iniciales,
    workers
)
```

Python enviará arreglos NumPy hacia C++.

C++ no definirá la red neuronal. La biblioteca C++ se encargará de serializar objetos de aplicación, fragmentarlos, enviar y recibir paquetes UDP, aplicar Go-Back-N, verificar CRC32, administrar ACK acumulativos, reconstruir objetos completos y devolver hacia Python los gradientes contenidos en los objetos `GRADIENT_RESULT`.

23. Posibles Mejoras Futuras

- Selective Repeat
- Timeout dinámico
- Multithreading
- Compresión de paquetes
- Balanceo dinámico de carga
- Aceleración GPU
- Paralelismo híbrido CPU/GPU

24. Tecnologías Utilizadas

Las tecnologías utilizadas son las siguientes:

Tecnología	Uso
Python	Definición y entrenamiento de la red neuronal
PyTorch	Machine Learning
C++	Biblioteca de comunicación UDP/RDT y serialización
UDP	Comunicación
pybind11	Integración Python/C++

25. Conclusión

El proyecto implementa un sistema de entrenamiento distribuido de una red neuronal utilizando UDP y una capa RDT desarrollada manualmente.

La arquitectura combina Python para la definición y entrenamiento de la red neuronal con una biblioteca C++ encargada de la comunicación confiable sobre UDP.

La confiabilidad será implementada mediante:

- Go-Back-N puro
- Ventanas deslizantes
- ACK acumulativos
- CRC32 aplicado sobre encabezado y payload
- Timeouts
- Retransmisiones automáticas
- Máximo de reintentos
- Estados definidos para emisor y receptor

El sistema permitirá dividir un único batch global entre 1 master y 10 workers, ejecutar el forward pass únicamente en el master, distribuir objetos `WORK_ASSIGNMENT` con los datos necesarios para backpropagation y combinar los gradientes recibidos en objetos `GRADIENT_RESULT` mediante promedio elemento a elemento.

La arquitectura propuesta servirá como base para futuras extensiones orientadas a entrenamiento paralelo de redes neuronales.